

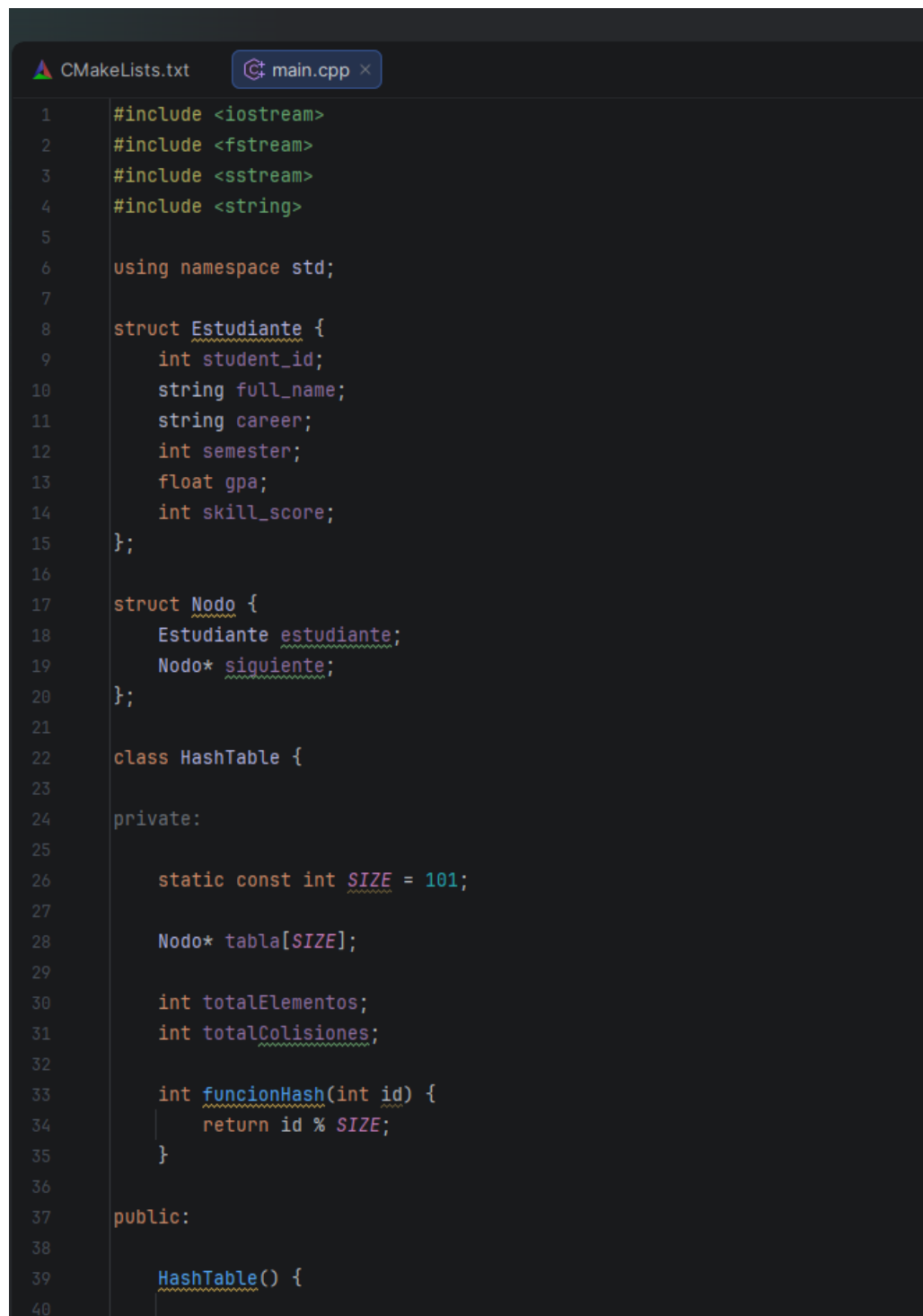
Esteban Javier Antonio Alarcón Herrera

9941-16-13183

Programación III

Tarea Tablas Hash

Captura del código completo C++:



```
1  #include <iostream>
2  #include <fstream>
3  #include <sstream>
4  #include <string>
5
6  using namespace std;
7
8  struct Estudiante {
9      int student_id;
10     string full_name;
11     string career;
12     int semester;
13     float gpa;
14     int skill_score;
15 };
16
17 struct Nodo {
18     Estudiante estudiante;
19     Nodo* siguiente;
20 };
21
22 class HashTable {
23
24 private:
25
26     static const int SIZE = 101;
27
28     Nodo* tabla[SIZE];
29
30     int totalElementos;
31     int totalColisiones;
32
33     int funcionHash(int id) {
34         return id % SIZE;
35     }
36
37 public:
38
39     HashTable() {
40
```

CMakeLists.txt

main.cpp x

```
22     class HashTable {
37     public:
39         HashTable() {
40
41             for (int i = 0; i < SIZE; i++) {
42                 tabla[i] = nullptr;
43             }
44
45             totalElementos = 0;
46             totalColisiones = 0;
47         }
48
49         bool insertar(Estudiante estudiante) {
50
51             int indice = funcionHash(estudiante.student_id);
52
53             if (buscar(estudiante.student_id) != nullptr) {
54
55                 cout << "Error: estudiante duplicado con ID "
56                     << estudiante.student_id << endl;
57
58                 return false;
59             }
60
61             Nodo* nuevo = new Nodo;
62
63             nuevo->estudiante = estudiante;
64             nuevo->siguiente = nullptr;
65
66             if (tabla[indice] != nullptr) {
67                 totalColisiones++;
68             }
69
70             nuevo->siguiente = tabla[indice];
71             tabla[indice] = nuevo;
72
73             totalElementos++;
74
75             return true;
76         }
```

```
22     class HashTable {
37     public:
49         bool insertar(Estudiante estudiante) {
76     }
77
78     Estudiante* buscar(int id) {
79
80         int indice = funcionHash(id);
81
82         Nodo* actual = tabla[indice];
83
84         while (actual != nullptr) {
85
86             if (actual->estudiante.student_id == id) {
87                 return &actual->estudiante;
88             }
89
90             actual = actual->siguiente;
91         }
92
93         return nullptr;
94     }
95
96     bool eliminar(int id) {
97
98         int indice = funcionHash(id);
99
100        Nodo* actual = tabla[indice];
101        Nodo* anterior = nullptr;
102
103        while (actual != nullptr) {
104
105            if (actual->estudiante.student_id == id) {
106
107                if (anterior == nullptr) {
108                    tabla[indice] = actual->siguiente;
109                }
110                else {
111                    anterior->siguiente = actual->siguiente;
112                }

```

```
sl CMakeLists.txt main.cpp x
22     class HashTable {
37     public:
96         bool eliminar(int id) {
103             while (actual != nullptr) {
105                 if (actual->estudiante.student_id == id) {
111                     anterior->siguiente = actual->siguiente;
112                 }
113
114                 delete actual;
115
116                 totalElementos--;
117
118                 cout << "Estudiante eliminado.\n";
119
120                 return true;
121             }
122
123             anterior = actual;
124             actual = actual->siguiente;
125         }
126
127         cout << "No se encontro estudiante con ese ID.\n";
128
129         return false;
130     }
131
132     void cargarCSV(string nombreArchivo) {
133
134         ifstream archivo(nombreArchivo);
135
136         if (!archivo.is_open()) {
137
138             cout << "Error al abrir el archivo.\n";
139
140             return;
141         }
142
143         string linea;
144
145         getline([&] archivo, [&] linea);
```

CMakeLists.txt

main.cpp x

```
22     class HashTable {
37     public:
132     void cargarCSV(string nombreArchivo) {
145         getline(&archivo, &linea);
146
147         int cargados = 0;
148
149         while (getline(&archivo, &linea)) {
150
151             stringstream ss(linea);
152
153             Estudiante e;
154
155             string temp;
156
157             getline(&ss, &temp, delim: ',');
158             e.student_id = stoi(temp);
159
160             getline(&ss, &e.full_name, delim: ',');
161
162             getline(&ss, &e.career, delim: ',');
163
164             getline(&ss, &temp, delim: ',');
165             e.semester = stoi(temp);
166
167             getline(&ss, &temp, delim: ',');
168             e.gpa = stof(temp);
169
170             getline(&ss, &temp, delim: ',');
171             e.skill_score = stoi(temp);
172
173             if (insertar(e)) {
174                 cargados++;
175             }
176         }
177
178         archivo.close();
179
180         cout << "CSV cargado correctamente.\n";
181         cout << "Se cargaron un total de " << cargados << " estudiantes." << endl;
```

```
CMakeLists.txt  main.cpp x
22     class HashTable {
37     public:
132         void cargarCSV(string nombreArchivo) {
184     }

185
186         void mostrarTabla() {
187
188             cout << "\n----- TABLA HASH ----- \n";
189
190             for (int i = 0; i < SIZE; i++) {
191
192                 cout << "\nBucket [" << i << "] -> ";
193
194                 Nodo* actual = tabla[i];
195
196                 if (actual == nullptr) {
197                     cout << "VACIO";
198                 }
199
200                 while (actual != nullptr) {
201
202                     cout << "\n----- \n";
203
204                     cout << "ID: "
205                         << actual->estudiante.student_id << endl;
206
207                     cout << "Nombre: "
208                         << actual->estudiante.full_name << endl;
209
210                     cout << "Carrera: "
211                         << actual->estudiante.career << endl;
212
213                     cout << "Semestre: "
214                         << actual->estudiante.semester << endl;
215
216                     cout << "GPA: "
217                         << actual->estudiante.gpa << endl;
218
219                     cout << "Skill Score: "
220                         << actual->estudiante.skill_score << endl;
221
222                     actual = actual->next;
223                 }
224             }
225         }
226     };
227 }
```

CMakeLists.txt

main.cpp x

```
22     class HashTable {
37     public:
186         void mostrarTabla() {
190             for (int i = 0; i < SIZE; i++) {
200                 while (actual != nullptr) {
222                     actual = actual->siguiente;
223                 }
224
225                 cout << endl;
226             }
227         }
228
229         void mostrarEstadisticas() {
230
231             cout << "\n----- ESTADISTICAS ----- \n";
232
233             cout << "Total de estudiantes: "
234                 << totalElementos << endl;
235
236             cout << "Total de buckets: "
237                 << SIZE << endl;
238
239             cout << "Total de colisiones: "
240                 << totalColisiones << endl;
241
242             cout << "Factor de carga: "
243                 << calcularFactorCarga() << endl;
244         }
245
246         float calcularFactorCarga() {
247
248             return (float) totalElementos / SIZE;
249         }
250
251         ~HashTable() {
252
253             for (int i = 0; i < SIZE; i++) {
254
255                 Nodo* actual = tabla[i];
256
257                 while (actual != nullptr) {
258                     delete actual;
259                     actual = actual->siguiente;
260                 }
261             }
262         }
263     };
264 }
```

CMakeLists.txt

main.cpp x

```
22     class HashTable {
37     public:
246         float calcularFactorCarga() {
247
248             return (float) totalElementos / SIZE;
249         }
250
251         ~HashTable() {
252
253             for (int i = 0; i < SIZE; i++) {
254
255                 Nodo* actual = tabla[i];
256
257                 while (actual != nullptr) {
258
259                     Nodo* temp = actual;
260
261                     actual = actual->siguiente;
262
263                     delete temp;
264                 }
265             }
266         }
267     };
268
269     int main() {
270
271         cout << "----- Esteban Alarcon -----" << endl;
272         cout << "----- 9941-16-13183 -----" << endl;
273         cout << "----- Tarea Tablas Hash -----" << endl;
274
275         HashTable hash;
276
277         hash.cargarCSV( nombreArchivo: 📁 "estudiantes.csv");
278
279         int opcion;
280
281         do {
282
283             cout << "\n----- MENU ----- \n";
```


CMakeLists.txt

main.cpp x

```
269     int main() {
281         do {
293             if (opcion == 1) {
295                 int id;
296
297                 cout << "Ingrese ID del estudiante: ";
298                 cin >> id;
299
300                 Estudiante* encontrado = hash.buscar(id);
301
302                 if (encontrado != nullptr) {
303
304                     cout << "\n----- ESTUDIANTE ENCONTRADO ----- \n";
305
306                     cout << "ID: "
307                          << encontrado->student_id << endl;
308
309                     cout << "Nombre: "
310                          << encontrado->full_name << endl;
311
312                     cout << "Carrera: "
313                          << encontrado->career << endl;
314
315                     cout << "Semestre: "
316                          << encontrado->semester << endl;
317
318                     cout << "GPA: "
319                          << encontrado->gpa << endl;
320
321                     cout << "Skill Score: "
322                          << encontrado->skill_score << endl;
323                 }
324                 else {
325
326                     cout << "No se encontro el estudiante. \n";
327                 }
328             }
329
330             else if (opcion == 2) {
331
```

CMakeLists.txt

main.cpp x

```
269     int main() {
281         do {
330             else if (opcion == 2) {
331
332                 int id;
333
334                 cout << "Ingrese el ID a eliminar: ";
335                 cin >> id;
336
337                 hash.eliminar(id);
338             }
339
340             else if (opcion == 3) {
341
342                 hash.mostrarTabla();
343             }
344
345             else if (opcion == 4) {
346
347                 hash.mostrarEstadisticas();
348             }
349
350             else if (opcion == 5) {
351
352                 cout << "Saliendo del programa\n";
353             }
354
355             else {
356
357                 cout << "Opcion no valida.\n";
358             }
359
360         } while (opcion != 5);
361
362         return 0;
363     }
```

Captura de ejecución C++:

```
Run untitled x
C:\Users\javie\Desktop\Universidad\Programacion III\Tarea Hash\cmake-build-debug\untitled.exe
----- Esteban Alarcon -----
----- 9941-16-13183 -----
----- Tarea Tablas Hash -----
CSV cargado correctamente.
Se cargaron un total de 3 estudiantes.

----- MENU -----
1. Buscar estudiante
2. Eliminar estudiante
3. Mostrar tabla hash
4. Mostrar estadísticas
```

```
Run untitled x
Ingrese ID del estudiante:1001

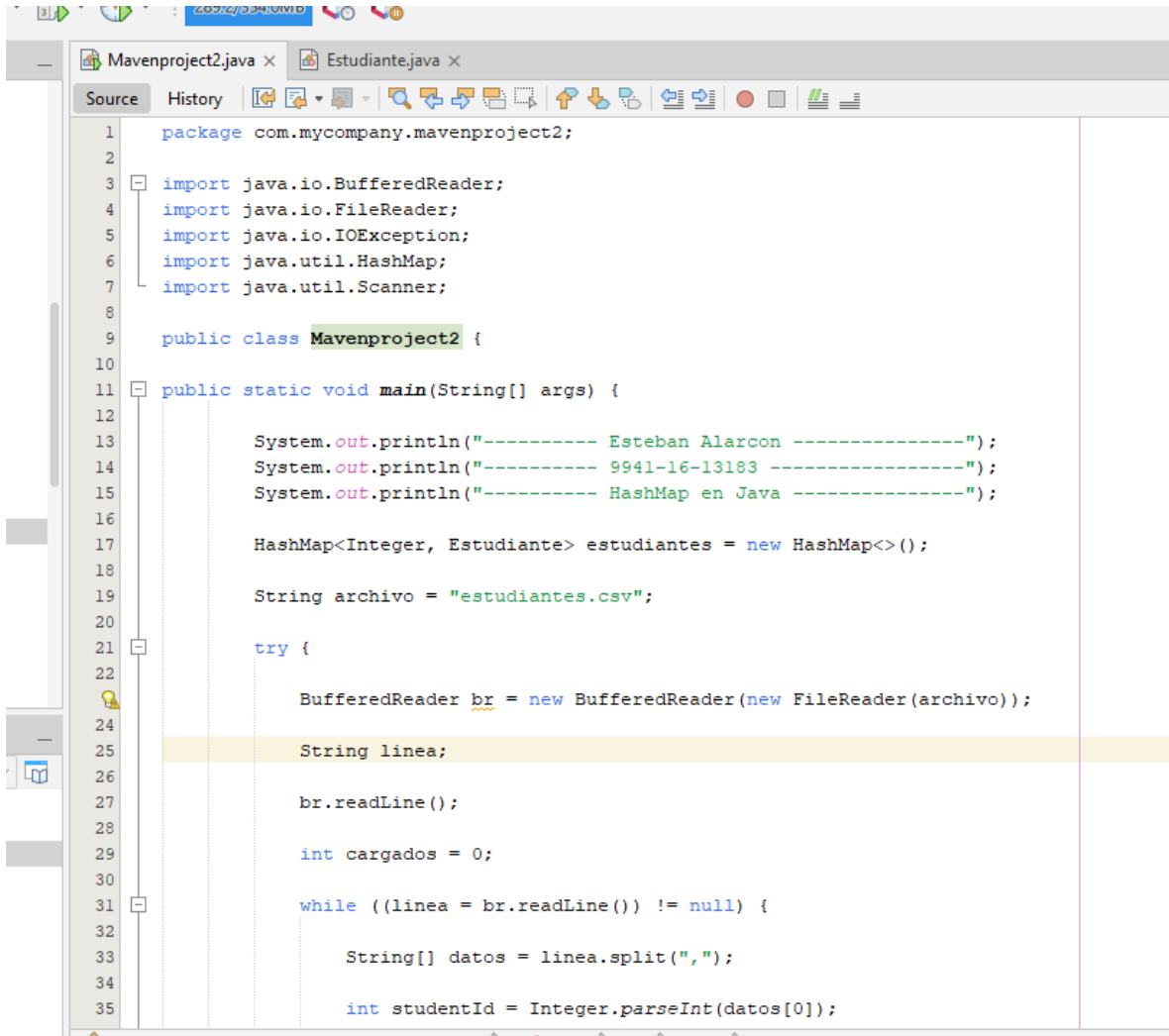
----- ESTUDIANTE ENCONTRADO -----
ID: 1001
Nombre: Juan Perez
Carrera: Ingenieria en Sistemas
Semestre: 5
GPA: 85.5
Skill Score: 120

----- MENU -----
```

```
4. Mostrar estadísticas
5. Salir
Seleccione una opcion:4

----- ESTADISTICAS -----
Total de estudiantes: 2
Total de buckets: 101
Total de colisiones: 0
Factor de carga: 0.019802
```

Captura del código completo Java:

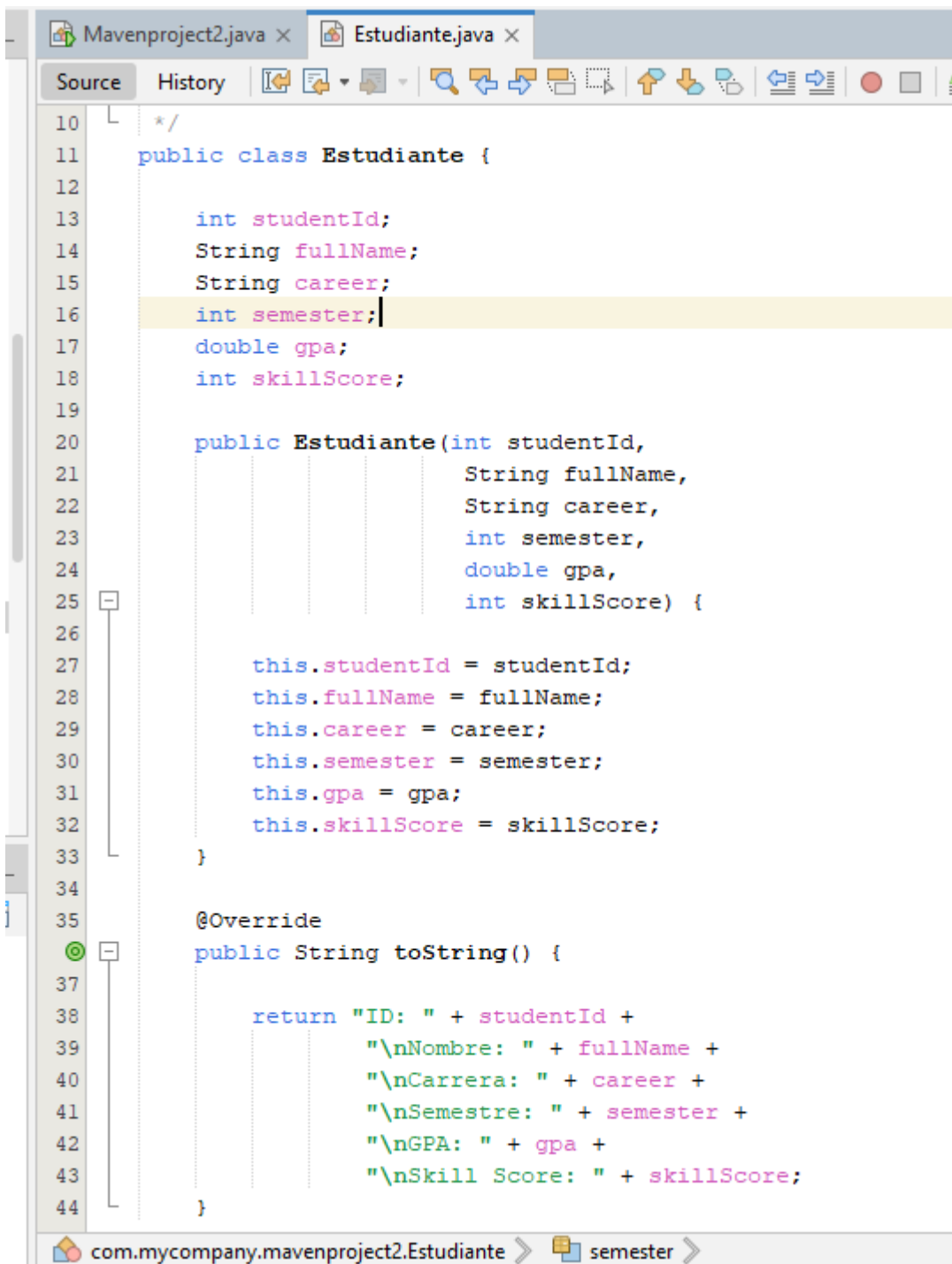


```
1 package com.mycompany.mavenproject2;
2
3 import java.io.BufferedReader;
4 import java.io.FileReader;
5 import java.io.IOException;
6 import java.util.HashMap;
7 import java.util.Scanner;
8
9 public class Mavenproject2 {
10
11     public static void main(String[] args) {
12
13         System.out.println("----- Esteban Alarcon -----");
14         System.out.println("----- 9941-16-13183 -----");
15         System.out.println("----- HashMap en Java -----");
16
17         HashMap<Integer, Estudiante> estudiantes = new HashMap<>();
18
19         String archivo = "estudiantes.csv";
20
21         try {
22
23             BufferedReader br = new BufferedReader(new FileReader(archivo));
24
25             String linea;
26
27             br.readLine();
28
29             int cargados = 0;
30
31             while ((linea = br.readLine()) != null) {
32
33                 String[] datos = linea.split(",");
34
35                 int studentId = Integer.parseInt(datos[0]);
```

```
Mavenproject2.java x Estudiante.java x
Source History
37 String career = datos[2];
38 int semester = Integer.parseInt(datos[3]);
39 double gpa = Double.parseDouble(datos[4]);
40 int skillScore = Integer.parseInt(datos[5]);
41
42 if (!estudiantes.containsKey(studentId)) {
43
44     Estudiante estudiante = new Estudiante(
45         studentId,
46         fullName,
47         career,
48         semester,
49         gpa,
50         skillScore
51     );
52
53     estudiantes.put(studentId, estudiante);
54
55     cargados++;
56 }
57
58
59 br.close();
60
61 System.out.println("\nCSV cargado correctamente.");
62 System.out.println("Total de estudiantes cargados: " + cargados);
63
64 } catch (IOException e) {
65
66     System.out.println("Error al leer el archivo.");
67 }
68
69 Scanner sc = new Scanner(System.in);
70
71 int opcion;
```

```
2964/5540MB
Mavenproject2.java x Estudiante.java x
Source History
73 do {
74
75     System.out.println("\n----- MENU -----");
76     System.out.println("1. Buscar estudiante");
77     System.out.println("2. Eliminar estudiante");
78     System.out.println("3. Mostrar estudiantes");
79     System.out.println("4. Mostrar estadísticas");
80     System.out.println("5. Salir");
81
82     System.out.print("Seleccione una opción: ");
83     opcion = sc.nextInt();
84
85     if (opcion == 1) {
86
87         System.out.print("Ingrese ID del estudiante: ");
88         int buscarId = sc.nextInt();
89
90         if (estudiantes.containsKey(buscarId)) {
91
92             Estudiante estudiante = estudiantes.get(buscarId);
93
94             System.out.println("\n=== ESTUDIANTE ENCONTRADO ===");
95             System.out.println(estudiante);
96
97         } else {
98
99             System.out.println("No existe estudiante.");
100         }
101     }
102
103     else if (opcion == 2) {
104
105         System.out.print("Ingrese ID a eliminar: ");
106         int eliminarId = sc.nextInt();
107     }
```

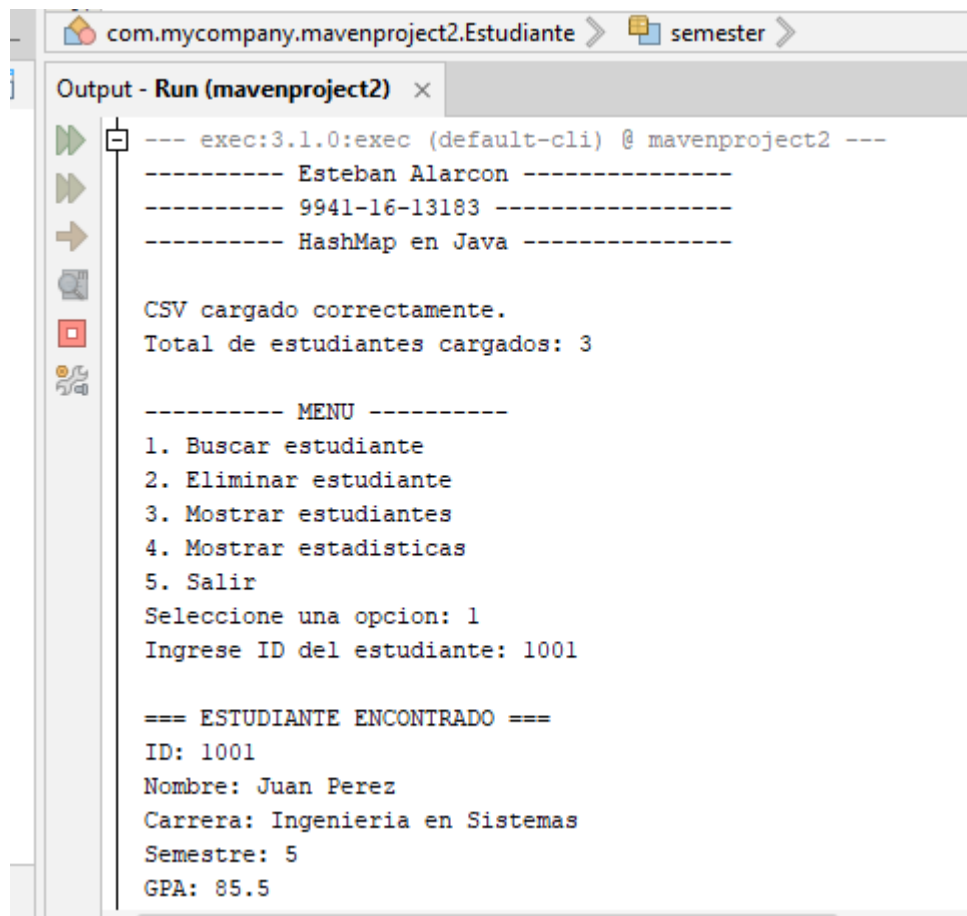
```
112         System.out.println("Estudiante eliminado.");
113
114     } else {
115
116         System.out.println("No existe estudiante para eliminar.");
117     }
118 }
119
120 else if (opcion == 3) {
121
122     System.out.println("\n=== ESTUDIANTES ACTUALES ===");
123
124     for (Integer key : estudiantes.keySet()) {
125
126         System.out.println("-----");
127         System.out.println(estudiantes.get(key));
128     }
129 }
130
131 else if (opcion == 4) {
132
133     System.out.println("\n=== ESTADISTICAS ===");
134     System.out.println("Total estudiantes: " + estudiantes.size());
135     System.out.println("Estructura utilizada: HashMap<Integer, Estudiante>")
136 }
137
138 else if (opcion == 5) {
139
140     System.out.println("Saliendo del programa...");
141 }
142
143 else {
144
145     System.out.println("Opcion no valida.");
146 }
147
148 } while (opcion != 5);
149
150 sc.close();
151 }
152 }
153
```



```
10  L  */
11  public class Estudiante {
12
13      int studentId;
14      String fullName;
15      String career;
16      int semester;
17      double gpa;
18      int skillScore;
19
20      public Estudiante(int studentId,
21                      String fullName,
22                      String career,
23                      int semester,
24                      double gpa,
25                      int skillScore) {
26
27          this.studentId = studentId;
28          this.fullName = fullName;
29          this.career = career;
30          this.semester = semester;
31          this.gpa = gpa;
32          this.skillScore = skillScore;
33      }
34
35      @Override
36      public String toString() {
37
38          return "ID: " + studentId +
39              "\nNombre: " + fullName +
40              "\nCarrera: " + career +
41              "\nSemestre: " + semester +
42              "\nGPA: " + gpa +
43              "\nSkill Score: " + skillScore;
44      }
```

com.mycompany.mavenproject2.Estudiante > semester >

Capturas de ejecución Java:



The screenshot shows an IDE's Run Output window for a project named 'com.mycompany.mavenproject2.Estudiante'. The output displays the execution of a Java application. It starts with a separator line, followed by the name 'Esteban Alarcon' and an ID '9941-16-13183'. A message 'HashMap en Java' is printed. Then, it reports 'CSV cargado correctamente.' and 'Total de estudiantes cargados: 3'. A menu is displayed with five options: '1. Buscar estudiante', '2. Eliminar estudiante', '3. Mostrar estudiantes', '4. Mostrar estadísticas', and '5. Salir'. The user selects option 1, enters the ID '1001', and the application finds the student. The found student's details are printed: 'ID: 1001', 'Nombre: Juan Perez', 'Carrera: Ingenieria en Sistemas', 'Semestre: 5', and 'GPA: 85.5'.

```
--- exec:3.1.0:exec (default-cli) @ mavenproject2 ---
----- Esteban Alarcon -----
----- 9941-16-13183 -----
----- HashMap en Java -----

CSV cargado correctamente.
Total de estudiantes cargados: 3

----- MENU -----
1. Buscar estudiante
2. Eliminar estudiante
3. Mostrar estudiantes
4. Mostrar estadísticas
5. Salir
Seleccione una opcion: 1
Ingrese ID del estudiante: 1001

=== ESTUDIANTE ENCONTRADO ===
ID: 1001
Nombre: Juan Perez
Carrera: Ingenieria en Sistemas
Semestre: 5
GPA: 85.5
```

Output - Run (mavenproject2) x

```
Ingrese ID del estudiante: 1001

=== ESTUDIANTE ENCONTRADO ===
ID: 1001
Nombre: Juan Perez
Carrera: Ingenieria en Sistemas
Semestre: 5
GPA: 85.5
Skill Score: 120

----- MENU -----
1. Buscar estudiante
2. Eliminar estudiante
3. Mostrar estudiantes
4. Mostrar estadisticas
5. Salir
Seleccione una opcion: 2
Ingrese ID a eliminar: 4
No existe estudiante para eliminar.

----- MENU -----
1. Buscar estudiante
2. Eliminar estudiante
```

```
----- MENU -----
1. Buscar estudiante
2. Eliminar estudiante
3. Mostrar estudiantes
4. Mostrar estadisticas
5. Salir
Seleccione una opcion: 4
```

```
=== ESTADISTICAS ===
Total estudiantes: 2
Estructura utilizada: HashMap<Integer, Estudiante>
```

----- MENU -----

1. Buscar estudiante
2. Eliminar estudiante
3. Mostrar estudiantes
4. Mostrar estadísticas
5. Salir

Seleccione una opción: 5

Saliendo del programa...

BUILD SUCCESS

Total time: 02:35 min

Finished at: 2026-05-20T21:10:50-06:00

|